image. Consider what might happen when you try to find the value of the pixel at row 10 and column 20 in the warped image. You pass the information: *row = 10, column = 20,* into your equations, and out pops: *comes-from row = 20.2, comes-from column = 14.5*. The point being, your calculations will likely use floating point, and therefore the comes-from addresses will not be integers. The easiest method to use is the **nearest neighbor** algorithm, that is, simply round the addresses to the nearest integer. This is simple, but can produce a very grainy appearance at the edges of objects where pixels may appear to be slightly misplaced.

Bilinear interpolation requires a little more effort, but provides significantly better images. Figure 23-16 shows how it works. You know the value of the four pixels *around* the fractional address, i.e., the value of the pixels at row 20 & 21, and column 14 and 15. In this example we will assume the pixels values are 91, 210, 162 and 95. The problem is to interpolate between these four values. This is done in two steps. First, interpolate in the *horizontal* direction between column 14 and 15. This produces two intermediate values, 150.5 on line 20, and 128.5 on line 21. Second, interpolate between these intermediate values in the vertical direction. This produces the bilinear interpolated pixel value of 139.5, which is then transferred to the warped image. Why interpolate in the horizontal direction *and then* the vertical direction instead of the reverse? It doesn't matter; the final answer is the same regardless of which order is used.

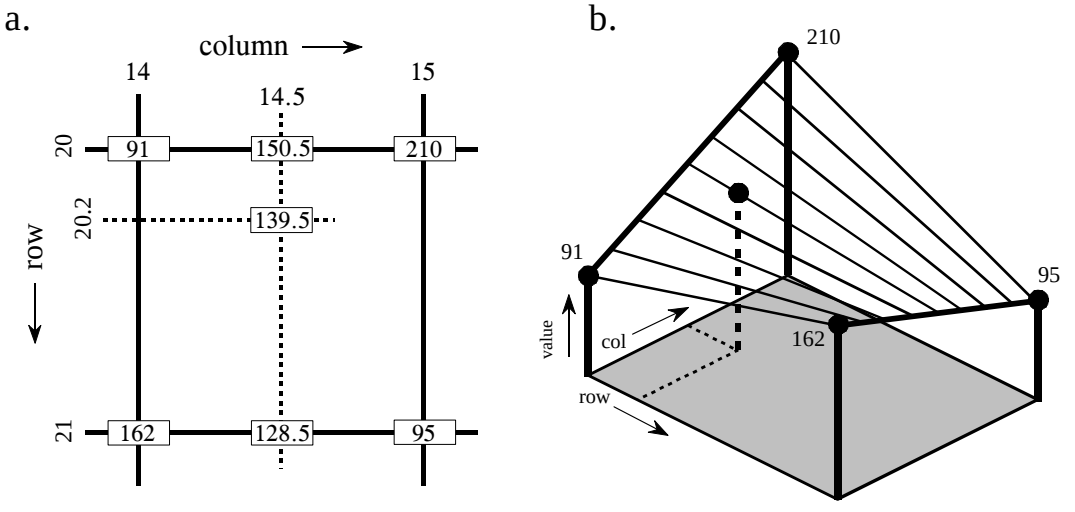

FIGURE 23-16
Subpixel interpolation. Subpixel interpolation for image warping is usually accomplished with bilinear interpolation. As shown in (a), two intermediate values are calculated by linear interpolation in the horizontal direction. The final value is then found by using linear interpolation in the vertical direction between the intermediate values. As shown by the three-dimensional illustration in (b), this procedure uniquely defines all values between the four known pixels at each of the corners.